

THE BUILD WIKI

The reference for the new nityastones.co.uk as it stands on 15 September 2026: what was built, how it works, how it ships, how it is checked, and what is still owed. Written for review once the deal is done and for use afterwards. The same pages live in the repository under `wiki/`.

SECTIONS

Start here

1. Overview

2. Architecture

3. Data model

4. Pages and features

5. Photographs and assets

6. Build and deploy

7. Audits

8. Integrations

9. Rules, decisions and open items

10. Glossary

11. History

12. Stock system

START HERE

Reference for the new nityastones.co.uk, written 14 September 2026 for review once the deal is done and for use afterwards. Everything here describes the `main` branch on that date. Where a fact is a decision still owed by the yard, it says so and points at `OPTIONS.md`.

PAGE	WHAT IT COVERS
1. Overview	What was built, what it is not yet, the numbers, where everything lives
2. Architecture	Stack, file map, the three builds, routing, state, themes, type
3. Data model	Products, packing and the quantity maths, prices, essentials, delivery bands, filters and search
4. Pages and features	Every route and what is on it; the order box, calculator, builder, 3D, gallery, pop-up, checkout calendar
5. Photographs and assets	The photo pipeline, manifests, studio renders, textures, logo, fonts
6. Build and deploy	Commands, the password gate, GitHub Pages, Cloudflare Pages, redirects, the path shim
7. Audits	Every check, what it catches, how to run it, the go-live audit
8. Integrations	WooCommerce, the stock tracker, the no-secrets rule, the field list
9. Rules, decisions and open items	The rules the site is built to, what the yard still owes, where those lists live
10. Glossary	The words used in the code and the documents
11. History	The build day by day, from the commit log
12. Stock system	The yard's own ledger: plan, seed, shared maths, the Worker API, phase status

The other documents in the repository, and what each is for:

- `HANDOVER.md` : for Coleisha (working on the site) and for the company (where things stand, who does what, what is needed).
- `DEPLOY.md` : the GitHub transfer, Cloudflare Pages, old-address redirects, go-live order, ownership.
- `OPTIONS.md` : every decision and asset still needed from the yard, in full.

- `COMPETITOR.md` : the Royale Stones study and what was taken from it.
- `UPDATES.md` : what changed after the owner meeting, separate from the pack.
- `web/AUDIT.md` : what has been checked, what was found, how to re-run.
- `web/PHOTOGRAPHY.md` : where better photographs come from, range by range.
- `present/` : the owner pack (PDF and source), the presenter script, the stock tracker brief, and a PDF copy of this wiki (`Nitya-Stones-Build-Wiki.pdf`).

1. OVERVIEW

WHAT IT IS

A complete new front end for nityastones.co.uk: a fast, single-page store built on the yard's own stock, prices and photographs, with an area-based ordering system, a patio builder, guides, projects, trade and contact pages, dark and light themes, and a 3D view of every stone. It runs behind a password gate on GitHub Pages for review, and is prepared to go live on Cloudflare Pages.

It replaces a WordPress/WooCommerce store whose listings contradict each other and whose pack sizes disagree with the warehouse. The new site takes its pack sizes from the warehouse database and its prices from the store, and computes everything else from those.

WHAT IT IS NOT YET

- **Connected.** It does not talk to WooCommerce or the warehouse stock tracker. The checkout stops at the last step. No payment is taken, nothing is stored on a server.
- **On the domain.** nityastones.co.uk still points at the old site.
- **Carrying the blog and legal pages.** The 45 blog posts, Terms, Privacy, Refunds, Delivery Terms and My Account stay on WordPress until moved; redirects for all of them are already written.

THE NUMBERS

RANGES	36 AS LISTED ON THE STORE, PLUS CLADDING AS 6 COLOURWAYS = 41 PRODUCT PAGES, PLUS SAMPLES
Essentials (jointing, primer, steps, edging, granite)	15 lines from the warehouse sheet, priced at the counter
Photographs	109 gallery frames, 23 project photographs, 36 studio renders, 36 textures, all from the yard's own images
Guides	9, with drawings to scale
Routes	69 crawled
Source	about 3,700 lines across 11 files in <code>web/src</code> ; the stock system adds a Worker, a schema, a shared package and a staff app under <code>wms/</code>
Commits	80, 10 to 15 September 2026
Gated page weight	about 1.1 MB to unlock; images and the 3D code load on demand

WHERE EVERYTHING LIVES

- Code: `github.com/feecrookz-bit/nitaya`, branch `main` . Every push rebuilds the preview by itself.
- Preview: `https://feecrookz-bit.github.io/nitaya/`, open since 15 September 2026 and kept out of search engines. A repository variable turns the password gate back on.
- Owner pack: `present/Nitya-Stones-Website-Upgrade-Pack.pdf` , 23 pages, with `present/SCRIPT.md` for the presenter.
- Stock tracker brief: `present/Nitya-Stones-Stock-Tracker-Link.pdf` .
- Private web copies of the site, the pack, the handover and the updates list exist as Claude artifacts; Fee holds the links.

WHO DOES WHAT

Fee built the site and does the go-live work (DNS, WooCommerce, the stock tracker) as the paid job. Once it is paid for, Nitya Stones owns everything on the site: code, design, graded photographs, documents. Upgrades and changes after that are arranged through Coleisha. Simar maintains the stock tracker sheet and script.

THE RULES IT IS BUILT TO

Nothing invented; only the yard's own photographs; the original logo; pack sizes only from the warehouse sheet; the wet view simulated and labelled; delivery bands marked indicative; no fee or charge the yard has not stated. Page 9 has the full list.

2. ARCHITECTURE

STACK

FRAMEWORK	REACT 19, ONE PAGE, A TINY HASH ROUTER WRITTEN IN <code>APP.JSX</code>
Build	Vite 8 (rolldown), <code>web/vite.config.js</code>
3D	three.js with react-three-fiber and drei, loaded lazily so the first paint never waits for it
Styling	One stylesheet, <code>web/src/index.css</code> , tokens at the top; no CSS framework
Type	Geist 400/500/700 and Cinzel 600, self-hosted from <code>web/src/fonts</code>
Lint	oxlint
Hosting	GitHub Pages (preview), Cloudflare Pages (live), both static
Backend	None. See page 8 for how WooCommerce and the stock tracker attach without one

No TypeScript, no state library, no router library, no component library. The whole site is eleven source files.

FILE MAP

nitaya/	
web/	the site
index.html	shell: meta, Open Graph, theme pre-paint, the path-to-hash shim
src/	
main.jsx	mounts App
App.jsx	every page and component (1,200 lines), the router, bag, theme
data.js	products, packing, prices, business details, delivery, filters, sea
content.js	long product copy: features, what is in the pack, laying, care
guides.js	the nine guides
Diagrams.jsx	the guides' drawings (SVG, to scale)
StoneScene.jsx	the 3D slab, laid field and wall
WetDry.jsx	the wet/dry slider
Motion.jsx	reveals, count-ups, marquee, parallax; all honour reduced-motion
Logo.jsx	the mark
index.css	the whole design
assets/	graded photographs, renders, textures, logo (21 MB, 211 files)
fonts/	
public/	favicon, touch icon, og.jpg, icons.svg, _redirects, _headers
scripts/	
photos.py	fetch, grade, gallery, textures (page 5)
photos.json, gallery.json	the two photo manifests
inline.mjs, protect.mjs	the password gate (page 6)
pages.mjs	the Cloudflare build
serve-pages.mjs	a local Cloudflare-style server
social.py	social post templates in the site's design
audit/	the audit scripts (page 7)
AUDIT.md, PHOTOGRAPHY.md	
present/	owner pack, presenter script, stock tracker brief
wiki/	this
.github/workflows/pages.yml	the preview deploy
HANDOVER.md DEPLOY.md OPTIONS.md COMPETITOR.md UPDATES.md README.md	

`index.html` at the repository root is the very first single-page prototype from 10 September and is not used.

THE THREE BUILDS

COMMAND (FROM <code>WEB/</code>)	OUTPUT	FOR
<code>BASE_PATH=/nitaya/ npm run build</code>	<code>dist/</code>	GitHub Pages preview, served under <code>/nitaya/</code>
<code>npm run pages</code>	<code>publish/</code>	Cloudflare Pages at the domain root (page 6)
<code>SINGLE=1 npm run build</code>	<code>dist-single/index.html</code>	One self-contained file with every image inlined; the audits and the pack screenshots run from it over <code>file://</code>

A fourth, `BASE_PATH=./ OUT_DIR=dist-artifact npm run build` , makes a relative-path build for the private web copy.

The default build puts the libraries in their own `vendor` chunk so the app chunk can be folded into the gated page while the lazy 3D chunk still finds React and three as plain files.

ROUTING

Addresses are hash routes: `/#/shop` , `/#/product/raj-green` , `/#/guide/laying-indian-sandstone` , `/#/shop?cat=indoor&colour=grey` . The router in `App.jsx` (`useRoute`) reads the hash, splits page, id, query and anchor, and scrolls to the top on change. Every page sets its own `<title>` from a `TITLES` map and the product and guide pages set structured data for search engines.

A path without the hash (`/product/raj-green`) is turned into the hash form by a short script in `index.html` before the app loads, so the redirects from the old store and hand-typed addresses both work.

STATE

Everything lives in memory and, for five things, in the browser's local storage under `nitya-` keys:

KEY	HOLDS
<code>nitya-bag</code>	The bag: line id and quantity per line
<code>nitya-designs</code>	Saved patio designs from the builder
<code>nitya-theme</code>	dark or light
<code>nitya-pc</code>	The last postcode typed into a delivery estimate
<code>nitya-offer</code>	When the offer pop-up last showed (it shows once a week)

Nothing is sent anywhere. The audits set `nitya-offer` before each run so the pop-up cannot intercept a click.

THEMES AND TYPE

Dark is the default; a sun/moon button in the header switches, and a two-line script in `index.html` applies the saved theme before the first paint so there is no flash. The light theme is a second token set under `:root[data-theme="light"]` , with its secondary text and kicker gold darkened until every text/background pair clears 4.5:1.

Type floor is 12 px anywhere on the site, 16 px for body text on phones. Tap targets are at least 44 px. Both are checked by script (page 7).

3. DATA MODEL

Everything a customer sees about a product comes from three files: `data.js` (the facts), `content.js` (the long copy) and the two photo manifests (page 5). There is no database.

A PRODUCT

Each range is one entry in `PRODUCTS`, created by a helper per family (`sand`, `indoor`, and plain `add` for the rest). The fields that matter:

FIELD	EXAMPLE	SOURCE
<code>id</code>	<code>raj-green</code>	Ours; it is the address of the product page and the key everywhere else
<code>slug</code>	<code>raj-green</code> or the store's slug	Names the studio render and texture files
<code>name</code> , <code>size</code> , <code>finish</code> , <code>cat</code>	Raj Green, Mixed patio pack, Riven, sandstone	The store's listing
<code>price</code> , <code>was</code>	19.50, 22.20 per m ² ex VAT	The store's current and struck prices
<code>unit</code>	per m ² , per pallet, per box, each	How the store sells it
<code>packing</code>	see below	The warehouse sheet
<code>cover</code> , <code>slabs</code> , <code>pack</code>	18.19, 60, "18.19 m ² per pack · 60 slabs in four sizes"	Computed from <code>packing</code>
<code>split</code>	true	Computed: every pack splits except boxes and cladding
<code>gallery</code> , <code>img</code> , <code>texture</code>	file lists	From the manifests, matched by id and slug
<code>content</code>	features, in the pack, laying, care	<code>content.js</code> , by id

Categories: sandstone, limestone, outdoor (20 and 16 mm porcelain), indoor (8 mm tiles), cladding. Cladding is six products sharing one content block and one packing, one per colourway photograph on the store's listing; their names are a decision for the yard.

PACKING: THE ONE TABLE THAT DRIVES THE MATHS

`PACKING` in `data.js` holds, per product id, what a pack physically is. Its values come from the "Database Products" tab of the stock tracker sheet and from nowhere else; never type a coverage figure anywhere else.

Two shapes:

- **Mixed** (the four 22 mm sandstone patio packs): sizes with counts and slab areas. 16 × 900 × 600, 16 × 600 × 600, 16 × 600 × 290, 12 × 290 × 290 = 60 slabs, 18.19 m².
- **Fixed** (`fixed(perPack, slabM2, perBox)`): one size. Kandla Grey 900 × 600 is 40 slabs of 0.54 m² = 21.60 m². Black Limestone 38 × 0.36. The 900 × 600 porcelains 40 × 0.54. The 16 mm 600 × 600 pallets 80 slabs in boxes of 2. Indoor 600 × 1200 tiles 64 a pallet in boxes of 2; 600 × 600 Brit Raven 160 in boxes of 4; 300 × 600 tiles 336 in boxes of 6 (Aspire Grey 280 in boxes of 5). Cladding 196 strips in boxes of 7.
- Sinai Pearl is not in the sheet; it is marked `assumed` at 50 slabs of 600 × 600 and listed for the yard to add.

From the table the code sets `cover` (m² per pack or pallet), the unit wording, and `split`. Indoor tiles and cladding are sold by the box, and the box is the unit; the pallet is only the delivery quantity.

FROM AN AREA TO WHAT LEAVES THE YARD

`quantify(product, m2)` is the single function behind the product page's by-the-area mode, the calculator, the builder, the bag and the checkout. It returns whole packs plus loose slabs:

- **Mixed packs:** whole packs, then the remainder made up in the pack's own four sizes by `mixFill`: first in the pack's proportions, then the last bit largest slab first, so the overshoot is never more than one small slab. 55 m² of Autumn Brown is 3 packs + 2 loose slabs = 55.01 m², not 4 packs = 72.76 m². If the loose set would be a whole pack it becomes one.
- **Single-size packs that split** (sandstone 900 × 600, limestone, the porcelains): whole packs plus loose slabs by the slab.
- **Boxes and kits:** whole units, rounded up.

Prices follow: `m2Price` normalises per-pallet prices to per m², `slabPrice` is a slab's share, and `exVat` prices a quantity as packs at the pack price plus loose slabs at the per-m² price. The bag and the checkout total by calling these, so they cannot disagree with the product page.

BAG LINE IDS

The bag stores line ids and quantities. The id says what the line is:

PREFIX	MEANING
<code>raj-green</code>	Whole packs of a product
<code>slabs:kandla-grey-900</code>	Loose slabs of a single-size pack
<code>mix:raj-green:1.1.0.0</code>	A loose set from a mixed pack, the four counts in the id, unit "per set"
<code>sample:raj-green</code>	A £5 sample
<code>ask:jointtec-pitch-black</code>	An Essentials line with no price: a quote request, priced at 0

`lineProduct(id)` in `App.jsx` turns any of these back into a displayable line with name, unit, coverage and price.

ESSENTIALS

ESSENTIALS: fifteen warehouse lines the store never listed. Joint-Tec in three colours, Por-Tec in four, primer, two step treads, two edgings, two granite setts and black cobbles, each with the sheet's per-pallet and per-box packing. They have no price on the site until the yard gives one; a customer can "Ask for a price", which puts a quote line in the bag. `essentialsFor(product)` picks the matching jointing compound and primer for the "Finish the job" strip.

DELIVERY

DELIVERY has three indicative bands by postcode area (local, London and M25, England and Wales) and a list of areas that are "ask us". `deliveryFor(postcode)` picks the band. The bands are marked indicative on every page until the real rates arrive. Free over £500 inside the M25 and 3 to 5 working days are the store's own published terms. `workingDaysFrom(3)` gives the earliest delivery day shown on the product page, bag and checkout.

FILTERS AND SEARCH

- Colour groups (`COLOUR_GROUPS` , `colour0f`): Blacks, Whites, Greens and multi, Browns and rust, Creams and buffs, Greys, from a per-product word list.
- Size (`size0f` , `SIZE_LABEL`): the two dimensions sorted so 900×600 and 600×900 are one chip.
- Finish (`finish0f` , `FINISH_LABEL`): riven, riven-effect, honed, matt, polished, split face, read from the finish text.
- Search (`SEARCH_TAGS`): colour and use words customers type that product names do not contain.

OTHER TABLES

`BUSINESS` (name, phones, WhatsApp, email, address, hours, socials, site) is the single source for every contact detail on the site; nothing else carries a phone number. `CATS` , `EDITIONS` , `SEASON` , `FAMILIES` and `PAIRS` arrange the same products into the home page's collections. `SCENES` is the 23 project photographs with captions. `PATTERNS` and `MIXED` describe the four laying patterns for the builder and the 3D field. `FAQ` and `REVIEWS` carry the store's own questions and its two reviews.

4. PAGES AND FEATURES

Every route, what is on it, and the components that do the work. All in `App.jsx` unless said otherwise.

ROUTES

ROUTE	PAGE	WHAT IS ON IT
<code>/</code>	Home	Slideshow hero with the mark and four facts; the drag-to-turn Raj Green slab; editions; this season's palette; offers; the builder teaser; sourcing declaration; families; guides; projects strip; newsletter; reviews
<code>/shop</code>	Shop	Every range; tabs by category and Offers; filter chips by colour, size and finish (in the address, so a filtered view can be shared); search from the header or menu via <code>?q=</code> ; editor's picks get the wide card
<code>/product/:id</code>	Product	Gallery with the 3D slide; unit table; the order box; See it in 3D; the story (features, in the pack, laying, care, honesty note); Finish the job; pairs with; FAQ
<code>/essentials</code>	Essentials	The fifteen warehouse lines, priced at the counter, Ask for a price
<code>/collections</code>	Collections	The editions and families as a browsing page
<code>/build</code>	Build your patio	Pick a stone, type width and length, pick a pattern, tick the allowance, see packs and price, save the design
<code>/guides , /guide/:slug</code>	Guides	Nine guides with drawings: porcelain or sandstone, laying sandstone, laying 20 mm porcelain, how many packs, sealing, cleaning, why slabs vary, the mixed pack, delivery day
<code>/projects</code>	Projects	23 of the yard's own photographs of laid stone, captioned by what is visible
<code>/trade</code>	Trade	A typical trade offer and a form; the wording is the yard's to confirm
<code>/samples</code>	Samples	£5 pieces of any range
<code>/about , /faq , /contact</code>		Yard, hours, map, the store's FAQ, the contact form
<code>/cart</code>	Bag	Lines with coverage, quantities, remove, delivery estimate, delivery-from date, collection
<code>/checkout</code>	Checkout	Contact, delivery or collection with the calendar, access notes, payment placeholder, summary; stops at the last step with the demo notice

Every page shows the header (logo, links, search on wide screens, bag, theme), the slide-out menu on narrow screens (with search and "WhatsApp the yard"), and the footer with every contact detail. The floating WhatsApp button appears on every page except checkout.

THE ORDER BOX (`OrderBox`)

At the top of every product page:

1. **Unit table:** per m², per slab (or "loose slabs by area" on mixed packs), per pack or pallet or box, each with the struck was-price.
2. **By the pack / By the area** toggle. Pack mode has a quantity stepper and, on single-size packs, a loose-slab stepper. Area mode takes m², a tick box for the allowance for cuts (5/10/15/20%), and says exactly what leaves the yard: "1 × 21.60 m² + 9 × 0.54 m² = 26.46 m²".
3. **Facts:** delivery from the earliest day, free over £500 inside the M25, collection from Mark Road with hours and a map link, the sample button.

4. Add to bag, which is also a sticky bar on phones.

THE CALCULATOR (**Calculator** , **AreaRows** , **CutAllowance**)

Several areas (width × length rows), the allowance tick box, and one result line: packs × coverage = m², with loose slabs listed by size for mixed packs. The same **CutAllowance** component is used in the builder and the order box, so the three always offer the same choices.

BUILD YOUR PATIO (**Build** , **Pattern** , **PatternThumb**)

Stone, dimensions, one of four patterns drawn to scale in SVG, allowance, whole packs and price, and a saved-designs list in local storage that can be re-added to the bag.

SEE IT IN 3D (**Stone3D** , **StoneScene.jsx**)

Two modes: one slab turned in the hand, or the stone laid in its pattern with pointed joints (a wall for cladding). Dry or wet. Everything is at true scale: a 22 mm slab is 22 mm thick against a 900 mm face. The face is the product's own texture (page 5) on a physical material that also takes its fine relief from the photograph as a bump map; sandstone and limestone get a cleft, flat-shaded top on top of that. Lighting is neutral with no tone mapping, calibrated so the top face shows the photograph's colour within a few percent, plus a small studio environment (drawn from light panels, no file fetched) for the reflections wet and polished stone need. The environment's share is set at scene level, because the material-level intensity does not scale it in this three.js. Wet is a darker base under a clear coat. The laid field gives each slab its own window on the texture, mirrored and rotated, plus a seeded tone shift (wider for natural stone than porcelain) and a slight height jitter for riven slabs, on a bed coloured as jointing compound. The cladding wall is individual 600 × 150 strips in a running bond, each with its own projection and its own band of the colourway photograph (the photograph is treated as about 1200 × 900 mm of built wall, its bottom edge skipped where the mark sits), so the six colourways build six different walls and the key light casts shadows between courses. The camera rig changes with the mode without remounting the canvas, which is what kept WebGL from losing its context. The wet look is labelled simulated.

GALLERY (**Gallery**)

Every photograph the store has for the range, swipeable, with a full-screen view rendered on the body so the header can never cover its close button. The first slide after the photographs is the turning slab, marked with a 3D badge on its thumbnail.

OFFER POP-UP (**OfferPopup**)

Once a week per browser, after nine seconds or when the mouse leaves for the tab bar. Built from the real was/now prices: the biggest saving and the count of ranges on sale. Never on the bag, checkout or builder; a dialog with focus inside it; Escape closes it.

CHECKOUT CALENDAR (**DayPicker**)

Earliest delivery is three working days out. The customer can keep "earliest" or pick a working day up to sixty days ahead; collection has the same choice with the yard's hours. The chosen day appears in the order confirmation. No slot fees exist because the yard has not stated any.

SEARCH

The header form on wide screens and the menu form on every screen both go to `/shop?q=`. The shop matches names, families, finishes and the `SEARCH_TAGS` vocabulary.

MOTION AND THEMES

`Motion.jsx` provides reveals, count-ups, the stone marquee and parallax; all stop under the system's reduced-motion setting. The theme button flips `data-theme` on the root and remembers the choice.

5. PHOTOGRAPHS AND ASSETS

Every image on the site is the yard's own, taken from its WordPress media library, its listings, its About page and its blog. No stock, no competitor imagery. The web view and the renders are made from those photographs and are labelled as such.

THE PIPELINE (`web/scripts/photos.py`)

COMMAND	DOES
<code>python3 scripts/photos.py fetch</code>	Downloads the originals listed in <code>photos.json</code> into <code>scripts/originals/</code> (not committed)
<code>python3 scripts/photos.py grade</code>	Grades each original and writes <code>src/assets/<name>.jpg</code>
<code>python3 scripts/photos.py gallery</code>	Builds the per-product gallery frames from <code>gallery.json</code>
<code>python3 scripts/photos.py textures</code>	Cuts a 1280 px face texture from each studio original for the 3D views; <code>face_box</code> in the manifest picks one clean slab where the photograph shows several
<code>python3 scripts/photos.py sheet</code>	A before/after contact sheet for checking the grade

The grade is the same pass for every image so the cards read as one shoot: grey-world white balance at 60%, auto-levels on the 0.5/99.5 percentiles, a gentle S-curve, +6% saturation, an unsharp mask, then a crop to the target aspect. A single tile on a plain ground is first cropped to the tile itself, which is what turns a phone snap on the yard floor into a product shot. Per-image overrides live in `photos.json` : `trim_top` and `trim_bottom` remove baked-in captions, `grade: false` skips the tonal work for texture maps.

THE MANIFESTS

- `scripts/photos.json` : every scene, hero, yard and studio image, with its source URL, target name, crop mode and overrides.
- `scripts/gallery.json` : the product galleries, in order, with captions.

To add a photograph: put its URL (or a file) in the right manifest with a `scene-` or `g-` name, run the command, and for a project photograph add a line to `SCENES` in `data.js` with the caption.

WHAT IS IN `src/assets`

PREFIX	COUNT	WHAT
<code>g-</code>	109	Gallery frames, per product
<code>scene-</code>	23	Project photographs
<code>p-</code>	36	Studio renders: one slab per range on a neutral ground, made from the yard's photographs
<code>t-</code>	36	Textures for the 3D views, cut from the renders
<code>hero</code> , <code>yard</code> , <code>pallets</code> , <code>slab</code>	4	The hero slideshow, the yard, the stock, the drag-to-turn slab
<code>logo</code>	2	The original mark, cleaned

PREFIX	COUNT	WHAT
payments	1	The card marks on the checkout

21 MB in all; the site loads them lazily and the gated preview fetches them on demand.

LOGO AND FONTS

The logo is the store's original, cleaned up, not redesigned. The site uses the 226 px web version upscaled; a vector is on the asks list for print and the favicon.

Geist 400, 500 and 700 and Cinzel 600 are self-hosted in `src/fonts`. The audit found the 700 file was a copy of the 500 and replaced it with the real bold; every weight used in the stylesheet maps to a loaded face.

BETTER PHOTOGRAPHS

`web/PHOTOGRAPHY.md` says, range by range, where studio-quality imagery comes from: the manufacturer's stockist packs for the branded porcelains, the factory catalogues for the Morbi tiles, and a half-day shoot at the yard for the quarry-named sandstones and limestones, with a shot list. The yard says it already has the photographs; the ask is for the originals rather than WhatsApp copies.

6. BUILD AND DEPLOY

The full step-by-step for the transfer and go-live is `DEPLOY.md`. This page explains the pieces.

COMMANDS (FROM `web/`)

<code>NPM CI</code>	INSTALL. <code>.NPMRC</code> CARRIES THE FLAG THE DEPENDENCIES NEED; <code>.NODE-VERSION</code> PINS NODE 22
<code>npm run dev</code>	Local site with live reload
<code>npm run build</code>	Static build to <code>dist/</code> ; <code>BASE_PATH=/nitaya/</code> for the GitHub preview
<code>SINGLE=1 npm run build</code>	One self-contained file to <code>dist-single/</code> for the audits
<code>npm run pages</code>	The Cloudflare build to <code>publish/</code>
<code>npm run preview:pages</code>	Serve <code>publish/</code> locally the way Cloudflare does, on port 8788
<code>npm run lint</code>	oxlint

THE PASSWORD GATE

Two scripts, both under "things not to touch" in the handover:

- `scripts/inline.mjs` folds the built page's own script and stylesheet into one HTML file, and deletes them from `dist/`, so nothing the gate protects is also published in the clear. Images, fonts and the lazy 3D chunk stay as hashed files and load on demand: about 1 MB to unlock rather than 25.
- `scripts/protect.mjs` encrypts that file with AES-256-GCM under a key derived from the password (PBKDF2-SHA256, 250,000 iterations, random salt). What is published is ciphertext plus a small unlock form; the browser derives the key and decrypts locally. There is no back door: a lost password means republishing with a new one. The unlock remembers for the tab, and a deep link survives the gate.

GITHUB PAGES (THE PREVIEW)

`.github/workflows/pages.yml` runs on every push to `main`: install, build under `/nitaya/`, inline, protect with the `SITE_PASSWORD` secret, add `404.html` and `.nojekyll`, publish to the `gh-pages` branch. If the secret is missing it stops with an error rather than publishing the site open. The repository variable `PUBLIC_PREVIEW=true` publishes it open on purpose, with a noindex tag; the preview has been open since 15 September 2026 (the owner's decision after the first payment). Remove the variable and the gate returns on the next deploy. A deploy takes about half a minute; the audit compares the size of the live gate page with the local build to confirm it landed.

CLOUDFLARE PAGES (THE LIVE SITE)

`scripts/pages.mjs` does the same job at the domain root, for Cloudflare to run on each push: gated when `SITE_PASSWORD` is set, open only when `PUBLIC_SITE=true`, and it refuses with neither. It writes no `404.html` on purpose: without one, Cloudflare serves the site with a 200 for any address, which the router and the path shim rely on.

Project settings: root directory `web`, build command `npm run pages`, output `publish`. Everything else is in the repository.

OLD ADDRESSES

`web/public/_redirects` maps every address on the current store: 40 products to their new pages, the four samples to Samples, categories to the filtered shop, about, contact, wholesale, FAQs, designer and services to their new pages, and the 45 blog posts, the legal pages and My Account to WordPress on a subdomain. The subdomain is written as `wp.nityastones.co.uk`, a working name to replace once decided.

Two of the old product addresses begin with an invisible character (`%e2%81%a0`); `DEPLOY.md` explains the check to run on the first deploy and the one-line fix if it fails.

`web/public/_headers` caches the hashed asset files for a year and sets the usual safety headers.

THE PATH SHIM

Five lines in `web/index.html` turn a path like `/product/raj-green` into the router's `/#/product/raj-green` before the app loads, keeping the query string. It is what makes the redirects and hand-typed addresses land on the right page.

OWNERSHIP

At the end the GitHub repository is in a company organisation, the Cloudflare account and DNS are the company's, and Fee and Coleisha are members of both. Nothing depends on a personal account.

`DEPLOY.md` section 6 has the table.

7. AUDITS

Tests are the definition of done here. Every check is a script in `web/scripts/audit/`, run from `web/`, and `web/AUDIT.md` records what each found and fixed. Most run against the single-file build (`SINGLE=1 npm run build`); the two that need the gate run against the gated build served locally.

BEFORE A PUSH

SCRIPT	CHECKS	NEEDS
<code>flows.cjs</code>	Behaviour: keyboard, focus, bag persistence, quantity edge cases, checkout validation, the ordering maths on the product page, calculator, builder, bag and checkout, the pop-up, business details on every page, structured data, route titles, the gate's failure paths	The gated <code>publish/</code> served at <code>http://localhost:8765/nitaya/</code>
<code>gate.cjs</code>	Unlock, wrong password, remembered session, deep link through the gate	Same
<code>typo.cjs</code>	Fonts loaded, weights covered, nothing under 12 px, every text colour from the token set, every text/background pair at 4.5:1 (3:1 large), both themes	dist-single
<code>axe.cjs</code>	axe-core accessibility over 17 routes in both themes; 0 violations is the bar	dist-single, <code>AXE=</code> path to axe.min.js
<code>crawl.cjs</code>	69 routes: no console errors, no failed requests, one h1, alt on every image, a label on every field	dist-single

BEFORE A DESIGN CHANGE GOES OUT

SCRIPT	CHECKS
<code>fit.cjs</code>	360, 400, 768, 1024, 1440 px: no horizontal overflow, no clipped text, tap targets at least 44 px
<code>overphoto.cjs</code>	Every line of text laid over a photograph, measured against the pixels behind it, both themes, 1440 and 400 px (<code>PNGJS=</code> path to pngjs)
<code>visual.cjs</code>	Contact sheets of 17 routes × 3 widths × 2 themes in <code>out/</code> , for a human look, plus two pages printed to PDF
<code>strings.mjs</code> + <code>spell.mjs</code>	Every visible string extracted and spell-checked (en-GB), and checked for consistency: m ² , x, en dashes, curly apostrophes, one spelling of the name

AT GO-LIVE

`golive.cjs` runs against the deployed site rather than a local build:

```
BASE=https://<project>.pages.dev node scripts/audit/golive.cjs
BASE=https://nityastones.co.uk EXPECT=open node scripts/audit/golive.cjs
```

It checks the gate is up or down as expected, the safety headers, that an unknown path serves the site with a 200, that hashed assets are cached immutable, the favicon and share image, every one of the 114 fixed redirects and the two wildcards, the two invisible-character addresses, and, on an open site, that old product addresses land on the right product page in a real browser and the checkout loads without

errors. It passes against the local Cloudflare-style server (`npm run preview:pages`) for both the open and the gated build.

RUNNING THE LOCAL SERVER FOR FLOWS AND GATE

```
mkdir -p /tmp/srv && ln -sf "$PWD/publish" /tmp/srv/nitaya  
(cd /tmp/srv && python3 -m http.server 8765 &)
```

The server dies between sessions; restart it before `flows.cjs` and `gate.cjs` . All the scripts set the offer pop-up's storage key before each run so it cannot intercept a click; the pop-up has its own test in flows.

WHAT THE AUDITS HAVE CAUGHT SO FAR

A synthesised bold (the 700 font file was a copy of the 500), a font weight with no face, text under 12 px in eleven places, kicker gold and secondary text failing contrast in the light theme, ink-on-ink headings in the dark panels, a white-on-white slider handle, gallery thumbnails with the wrong role, heading jumps, pages without an h1, unlabeled fields, a class name clash between the mixed-pack hint and the delivery estimate, the 3D badge losing contrast through a dimmed thumbnail, and the WhatsApp button outside any landmark. All fixed; the log is in `web/AUDIT.md` .

8. INTEGRATIONS

The site has no backend and holds no secrets. The two systems it must work with, and how, without changing that.

THE RULE

Anything in the front end is readable by anyone who opens the page. So the WooCommerce key and the stock tracker's shared secret never go in the site's code, and the design avoids needing them in the browser at all.

WOOCOMMERCE: THE TILL

WooCommerce stays as the system of record for orders and payments, on a subdomain after go-live.

- The site reads products, stock and creates the cart through WooCommerce's **Store API**, which is built to be called from a browser without a key.
- Payment happens on the gateway's hosted form (Stripe, PayPal, Klarna or WooPayments, whichever the store has). The checkout page in the demo has the same fields and hands over at the last step.
- What Fee needs from the company: a REST API key (for setup and the stock sync, not for the browser), the payment plugins and their test keys, shipping zones and rates, an admin or staging login. All listed in `HANDOVER.md`.

THE STOCK TRACKER: THE WAREHOUSE

A Google Sheet with an Apps Script, maintained by Simar, with roles for Admin, Director, Warehouse and Office. He changes it regularly, so the connection is to a fixed doorway, not to his script.

- **Orders out:** WooCommerce's built-in webhook posts each paid order to the script's web app. No hosting on our side.
- **Stock in:** the script runs on a timer and writes available quantities into WooCommerce's stock field using the REST key, which lives in the script's properties. The site then reads stock from WooCommerce like everything else.
- **What stays fixed:** the web app address (deploy once, publish new versions to the same deployment), the shared secret, the two entry point names, and the field names. Everything else in the sheet and script is Simar's to change; a Changelog tab announces changes.
- **The field list** is in `present/Nitya-Stones-Stock-Tracker-Link.pdf`: read stock (code, name, size, thickness, base unit, mode, per pallet, per box, m² per unit, available, active), record order (order number, placed at, customer, fulfilment, day, postcode, lines with code, quantity, unit and slab-size counts for a split mixed pack, notes), and the reply (ok or a reason the site can show).
- **First job:** a product code on every row of the Database Products tab, matching the WooCommerce SKU. The sheet has no codes today, and no quantity column; both are decisions in that brief.

SINAI PEARL AND THE CLADDING

Sinai Pearl is missing from the sheet; the six cladding colourways are one row. Both need rows with codes before the sync can carry them.

ORDER OF CONNECTION

1. Simar's test copy of the sheet and script, with made-up stock.
2. The site's `pages.dev` address pointed at a WooCommerce staging copy; a test order end to end.
3. Live sheet and live WooCommerce.

Fee does this work as part of the build, up to and including go-live, and is paid for it. Once the build is paid for, Nitya Stones owns everything on the site. Upgrades and changes after that are arranged through Coleisha.

9. RULES, DECISIONS AND OPEN ITEMS

THE RULES THE SITE IS BUILT TO

These are why the demo is trusted, and they bind every future change:

1. **Nothing invented.** No reviews, figures, awards, "since" dates, delivery promises, fees or claims that did not come from the yard or its current site. An unconfirmed fact goes in `OPTIONS.md`, not on a page.
2. **Only the yard's own photographs.** Never a competitor's, never stock. Renders and 3D views are made from the yard's photographs and say so.
3. **The original logo**, cleaned, not redesigned.
4. **Pack sizes from the warehouse sheet only**, through `PACKING` in `data.js`. Never a coverage figure typed by hand.
5. **The wet look is simulated** and labelled until the hosed photographs are in.
6. **Delivery bands are indicative** until the real rates arrive; free over £500 inside the M25 and 3 to 5 working days are the store's own terms.
7. **The offer pop-up** reads the real was/now prices, never shows on the bag, checkout or builder, once a week, Escape closes it. Not a discount code the yard has not agreed.
8. **Text over photographs stays readable**, measured; nothing under 12 px.
9. **Both themes, three widths** checked before anything visual is pushed.
10. **No secrets in the front end.**
11. **The gate scripts, the workflow and the photo script** are not edited by hand.

WHAT THE YARD STILL OWES

The full list with detail is `OPTIONS.md`; the short form is `HANDOVER.md` "What we need". In three groups:

- **Access (for Fee):** WooCommerce REST key; payment plugins and test keys; shipping rates; a WordPress admin or staging login; view access to the sheet and the script's web app URL; DNS access nearer the day.
- **Answers (for Coleisha to put on the site):** cladding colourway names; delivery terms today and any split-pack or cutting charge; the copy confirmations ("since 2016", price match, own quarries, £5 samples, what a trade account includes); the five store listings to correct so they match the sheet; Sinai Pearl added to the sheet; whether WooCommerce stays as the till; the WordPress subdomain name.
- **Assets (for Coleisha):** the photograph originals; the logo as a vector; a reviews link; prices for the fifteen Essentials lines.

DELIBERATELY LEFT ALONE

From the Royale Stones study (`COMPETITOR.md`): showrooms, the £80 split-pallet fee and other fees, a paid cutting service, Trustpilot and its reviews, size variants in a dropdown, bathrooms and pergolas, quantity discounts at checkout, a price-range slider. Each waits on the yard's own answer, not on a build.

DECISIONS ALREADY MADE

- The mixed patio pack is sold honestly: the pack is $16/16/16/12 = 18.19 \text{ m}^2$ and the calculator never rounds it away; every pack splits.
- Cladding is six products, one per colourway photograph.
- Dark is the default theme; the owner can choose light.
- WooCommerce stays as the till, with the sheet syncing to it, rather than the site talking to the sheet.
- Cloudflare Pages for the live site, GitHub Pages for the preview; no Vercel.
- The blog moves under Guides with redirects rather than being dropped.
- The preview is open (15 September, after the first payment), with a noindex tag; the gate can be restored by removing one variable.
- The yard gets a stock system of its own rather than waiting on the tracker; the tracker's data seeds it, and its maintainer's remaining jobs need no code.

10. GLOSSARY

TERM	MEANING
Range	One product as the store lists it: a stone or tile in one size and thickness. 36 on the store, 41 product pages on the site
Pack	The unit the yard sells natural stone in: a crate of slabs. Mixed packs hold four sizes; single-size packs one
Pallet	The delivery unit for porcelain and tiles. On the site "pack" and "pallet" are the same quantity; the word follows the category
Box	The sales unit for indoor tiles and cladding; the pallet is only the delivery quantity
Cover	m ² per pack, pallet or box, computed from the packing
Split	Selling part of a pack: loose slabs. Every pack splits except boxes and cladding
Loose slabs	Slabs sold outside a whole pack; for mixed packs, a set in the pack's four sizes
Mixed patio pack	22 mm sandstone: 16 × 900 × 600, 16 × 600 × 600, 16 × 600 × 290, 12 × 290 × 290 = 18.19 m ²
Allowance for cuts	The extra ordered for cutting and breakage, 5/10/15/20%, a tick box, never silently added
Essentials	The warehouse's jointing compounds (Joint-Tec, Por-Tec), primer, step treads, edging, granite setts and cobbles
Ask for a price	An Essentials line put in the bag as a quote because the yard has not given a price
Edition, family, season	Ways the home page arranges the same products; nothing is sold differently
Studio render	A slab on a neutral ground made from the yard's photograph; the <code>p-</code> files
Texture	The square crop of a render used by the 3D views; the <code>t-</code> files
Scene	A project photograph of laid stone; the <code>scene-</code> files
Gate	The password page in front of the preview; the site is encrypted behind it
Single-file build	The one-HTML-file build used by the audits and the pack screenshots
Hash route	An address of the form <code>/#/page/id</code> ; the site has no server-side pages
Path shim	The script that turns <code>/product/x</code> into <code>/#/product/x</code>
Redirects file	<code>_redirects</code> : the signposts from every old store address to the new site
The sheet	"stock tracker main", the Google Sheet and Apps Script that is the warehouse's stock system; its "Database Products" tab is the authority on pack sizes
Store API	WooCommerce's browser-safe API for products, cart and checkout, needing no key
The pack	The owner presentation pack (PDF), separate from a pack of stone
Fee, Coleisha, Simar	Fee built the site and does DNS and the connections; Coleisha takes over the site; Simar maintains the sheet

11. HISTORY

80 commits over six days, from the commit log.

10 SEPTEMBER: FROM A PAGE TO A STORE

A single-page prototype with drawn swatches and no photographs. The same day: a React rebuild with the 3D sandstone hero; the yard's own photography fetched from its media library and graded with a repeatable pass; the full catalogue; a rebuild as a store with shop, product pages, bag and checkout, studio renders and the dark design; editions, the patio builder and stone families; guides with drawings; the social template generator; the GitHub Pages deploy.

11 SEPTEMBER: MOTION AND THE GATE

The cinematic hero, marquee, count-ups and reveals. The password gate: the single-file build encrypted behind an unlock page, made to hold until the site frame has loaded, and to refuse an open deploy without the secret. The wet/dry slider, the delivery estimate, lazy 3D, guide diagrams, smarter search, bag coverage.

12 SEPTEMBER: THE DESIGN UPGRADE

One type system, self-hosted fonts, the taller hero, banners, pairings, the sticky buy bar, a fuller About. Dark theme as a second token set with a switch. Trade, Projects and Offers. The original logo restored everywhere. Header fixes for phones: the drawer, logo aspect, tap targets, hero text on pale photographs.

`OPTIONS.md` written.

13 SEPTEMBER: EVERY PHOTOGRAPH, THEN THE AUDIT

Every photograph the store has, on the product pages, with the detail from the listings. The full-screen gallery. The stone at scale on home and shop. The launch audit: fonts, copy, business details, accessibility, behaviour, print, with the fixes. The owner pack with the current site beside the new one, and the presenter script.

14 SEPTEMBER: THE MEETING AND AFTER

The seven customer photographs from About and nine more from listings and the blog. Captions over photographs made readable. Then the ordering rework: by the pack or by the area, loose slabs, the unit table, delivery dates, the offer pop-up, pack sizes from the warehouse database. See it in 3D on every product; cladding as six colourways; the 3D slide in the gallery. Pack sizes settled by the sheet. The handover. The allowance for cuts as a tick box. Essentials. The checkout calendar. Every pack splits, mixed packs in their four sizes. `UPDATES.md`. Shop filters, header search, the WhatsApp button. The stock tracker brief for Coleisha and Simar. Hosting and ownership in the handover. The Cloudflare preparation: build script, redirects, headers, path shim, `DEPLOY.md`. The go-live audit and this wiki.

15 SEPTEMBER: READINESS, THE STONE MADE REAL, THE STOCK SYSTEM

The readiness audit with the end-to-end loop, and its fixes (touch targets, small print, the WhatsApp button off the captions, footer headings). The 3D stone rebuilt to match the stone sold: true thickness, neutral light, textures re-cut; then the laid field and the cladding wall. The password rotated, then the preview opened after the first payment. The stock system: plan, seed from the warehouse sheet, shared pack maths, the Worker over D1 with sign-in and roles, the first staff screens, the import for Simar's files,

the notes and steps for him and his AI. The backend request, the one-page status with the time scale, the wiki and its PDF.

12. STOCK SYSTEM

The yard's own stock system, planned on 15 September 2026 when the existing tracker was slow to open up. The full plan is `wms/PLAN.md` ; this page is the map and the status.

WHAT EXISTS

PART	WHERE	STATE
Plan	<code>wms/PLAN.md</code>	Written: jobs, principles, architecture, codes, data model, screens, phases, decisions
Seed	<code>wms/seed/products.csv</code>	The warehouse sheet with a code per line: 109 rows, 72 active codes, 11 flagged
Shared pack maths	<code>wms/shared</code> (<code>@nitya/packing</code>)	<code>coverOf</code> , <code>mixFill</code> , <code>quantify</code> , <code>packFrom</code> ; nine tests pass
API	<code>wms/api</code>	Cloudflare Worker over D1: schema, seed migration, Access sign-in with roles, products, quantify, stock, users, public availability
Staff app	<code>wms/app</code>	First screens: Today, Stock, a product (packing, stock, the area calculator, movements), Users. Builds; runs against the local Worker
Import	<code>wms/api/scripts/import.mjs</code>	Turns an opening count and a transactions export into ledger movements; refuses a file with a problem and names the lines. Sample files in <code>wms/seed/samples</code>
For Simar	<code>wms/SIMAR.md</code> , <code>wms/SIMAR-AI.md</code> and their PDFs in <code>present/</code>	The features note with the eleven flagged rows, and the four tasks written for his AI with exact file formats

PHASE 0: DONE

Acceptance (`wms/api/scripts/smoke.mjs` , against the local Worker): every seed row in the database (88 products, 78 active, 48 matched to site products, 36 mixed-pack components), the Raj Green mixed pack covers 18.19 m² in 60 slabs, 55 m² of Autumn Brown is 3 packs + 2 loose slabs = 55.01 m² (the same answer the site gives), 26.4 m² of Quartz White is 1 pallet + 9 slabs, the stock view answers for every active product, the public availability route answers by site id, an admin can list users, an unknown route is 404.

Two things learned on the way, both now in the code:

- The sheet rounds m² per slab to two decimals (0.17 for 600 × 290, 0.08 for 290 × 290), which shrinks a mixed pack to 18.08 m². The seed computes m² from the stated dimensions instead, exact to the millimetre, and uses the sheet's figure only where there are no dimensions.
- Wrangler environments do not inherit bindings; the dev environment declares the database again.

WHERE IT LIVES

`admin.nityastones.co.uk` : the staff app at the root (a Pages project with that custom domain), the Worker under `/api/*` (a route in `wrangler.toml`), one Cloudflare Access application over the hostname so both share the sign-in. It needs the domain's DNS on Cloudflare; until then the `pages.dev` and `workers.dev` addresses stand in. DEPLOY.md section 3a has the steps.

HOW IT FITS

The site keeps its own product file for copy and photographs; the stock system is the authority on packing and availability. `site_id` on a product row ties the two. The site's `PACKING` table and the seed agree today; when the shared package is imported by the site (a small change, later), there is one source.

WooCommerce is not in the system. When it joins, its order webhook posts to the web-order route and the Worker pushes availability into its stock field, both on the same Worker.

WHAT SIMAR OWES

No code. The flagged-row decisions, an opening stock count in base units on one day, the code column on his sheet, and optionally an export of his transactions. His AI has the steps and the formats; our import checks and loads the files.

NEXT

Phase 1: the movements routes (goods in, adjustment, count, transfer, breakage) and the first screens of the staff app (Today, Stock, Goods in, Counts). Then orders, deliveries, the site link, notifications and reports, as the plan sets out.